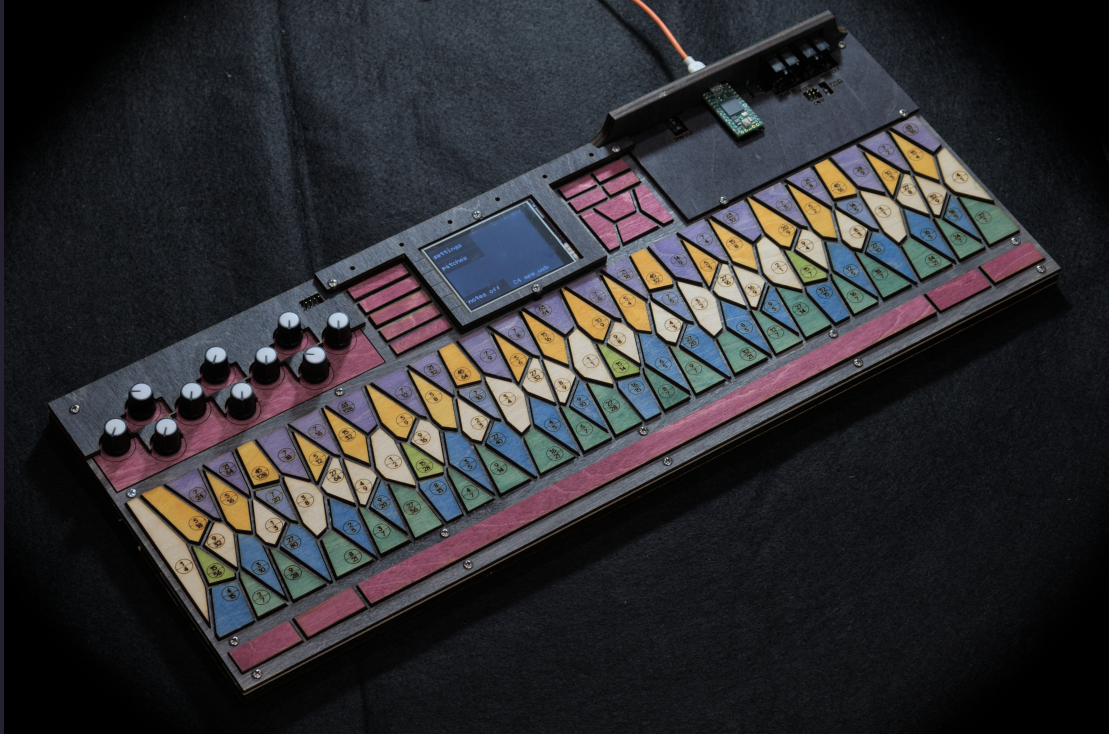


# Seven Limit Mosaichord User Manual



Copyright 2024-2025 Jim Snow

## Contents

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>Description</b>                                               | <b>4</b>  |
| <b>Music, Math, Just Intonation, and Navigating the Keyboard</b> | <b>4</b>  |
| <b>Prime Limit</b>                                               | <b>5</b>  |
| <b>Scales and Chords</b>                                         | <b>6</b>  |
| <b>Basic Operation</b>                                           | <b>11</b> |
| <b>Knobs</b>                                                     | <b>14</b> |
| <b>Menus</b>                                                     | <b>15</b> |
| <b>Using the Internal Synthesizer</b>                            | <b>17</b> |
| <b>Using an external synthesizer</b>                             | <b>18</b> |
| <b>MPE Synths</b>                                                | <b>19</b> |
| Surge XT . . . . .                                               | 19        |
| <b>Multitimbral MIDI Synths</b>                                  | <b>20</b> |
| E-mu Proteus 2000 . . . . .                                      | 22        |
| Korg SP-300 . . . . .                                            | 23        |
| Roland RD-300NX . . . . .                                        | 25        |
| Yamaha FB-01 . . . . .                                           | 25        |
| Yamaha MOX8 . . . . .                                            | 26        |
| <b>MIDI → CV devices</b>                                         | <b>26</b> |
| Arturia Keystep Pro . . . . .                                    | 26        |
| Befaco MIDI Thing V2 . . . . .                                   | 27        |

|                                                                   |               |
|-------------------------------------------------------------------|---------------|
| <b>Mono Synths</b>                                                | <b>29</b>     |
| Moog Slim Phatty . . . . .                                        | 29            |
| <b>Tuning Table Synths</b>                                        | <b>29</b>     |
| Yamaha DX7 with Grey Matter Response E! expansion board . . . . . | 30            |
| Dexed . . . . .                                                   | 31            |
| <br><b>Firmware updates and modification</b>                      | <br><b>32</b> |
| Install/Setup development tools                                   | 32            |
| Getting Source Code                                               | 32            |
| Debugging                                                         | 33            |
| <br><b>Assembly and Disassembly</b>                               | <br><b>34</b> |
| <br><b>Theory of Operation</b>                                    | <br><b>36</b> |
| Keys                                                              | 37            |
| Multiplexing inputs                                               | 37            |
| Autocalibration . . . . .                                         | 40            |
| Generating key pressure from MPE                                  | 41            |
| Other Components                                                  | 41            |
| <br><b>Hardware Revisions</b>                                     | <br><b>42</b> |
| <br><b>Troubleshooting</b>                                        | <br><b>44</b> |

# Description

The Seven Limit Mosaichord is an expressive musical keyboard controller with four octaves of pressure-sensitive keys, designed around a tuning system called just intonation, in which the frequencies of musical notes are related to each other by whole-number ratios. This makes it possible to play many musical intervals that align cleanly with the harmonic series, rather than rounded to the nearest pitch that lines up with the standard 12-tone octave “equal temperament” (12-TET) system used for most conventional modern music.

The 28-note-per-octave scale includes 16 “5-limit” tones that serve a recognizably similar role as the regular 12 tones, plus it has 12 “7-limit” tones, most of which have no close 12-TET equivalent. The Seven Limit Mosaichord spans four octaves and has 113 keys in total.

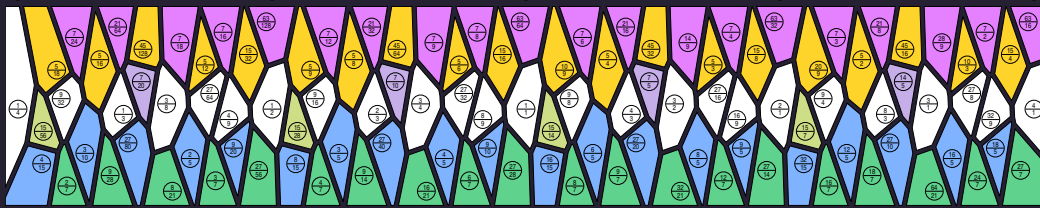
The Mosaichord works best with an MPE-capable synthesizer, but we also support a variety of older pre-MPE multi-timbral synthesizers as well by playing each concurrent note on a separate MIDI channel.

The Mosaichord also has its own audio DAC and a built-in synth engine following a fairly simple subtractive synthesis design (basic waveforms into a low-pass filter).

MIDI 2.0 is not currently supported.

The Mosaichord is powered over USB, and the default configuration is to send MPE MIDI over USB, to be used by an appropriate software synth. (Surge XT is a good zero cost open source option.) It also has the conventional DIN5 MIDI in and MIDI out ports, to be used with synths that use those.

## Music, Math, Just Intonation, and Navigating the Keyboard



As the Seven Limit Mosaichord keyboard layout is unique to this instrument and is based

on just intonation (JI), a tuning system that is usually treated as a theoretical curiosity or historical footnote if it is mentioned at all outside of a small community of tuning system enthusiasts, it is necessary at this point to digress for a moment and talk about the mathematical foundations of musical sound and how they relate to the idiosyncratic shapes, physical arrangement, and colors of the keys.

In just intonation, the frequency of every note is related to all the others by some whole-number ratio. In a sense, just intonation is the most basic, universal tuning system there is, as sounds that form whole number ratios are what we perceive as being "in tune". Why this is so has to do with the harmonic series of each note – the sounds we normally hear in music or just everyday life are not usually pure sine waves, but rather can be thought of as a collection of overlapping sine waves called harmonics (or overtones) at multiples of the base "fundamental" frequency. Thus a sound with a base frequency of 200 hz will usually also be accompanied by sound energy at 400 hz, 600 hz, 800 hz, and so on. (Some "inharmonic" sounds don't follow this rule, but most musical sounds do – or at least try to come as close as they can.)

If you were to play another sound concurrently at 300 hz, its second harmonic at 600 hz lines up with the 200 hz sound's third harmonic also at 600 hz, and that sounds very in-tune to us. If you detune, say, the 300 hz sound to 299 hz, then its second harmonic at 598 hz no longer lines up with the other note's harmonic at 600 hz, and it produces an audible 2 hz warble, called a beat frequency or a difference tone as they go back and forth between contrstructively and destructively interfering with each other. We tend to perceive this phasing-in-and-out unstableness in musical intervals as being out of tune. A little bit of out-of-tuneness is tolerable and sometimes desirable as a chorus effect, but usually we'd rather avoid it if we can when it isn't deliberate.

Sounds that are related to each other by small whole-number ratios generally sound the most consonant together, and the larger, more complex ratios sound increasingly dissonant and risk falling into unintelligibility, especially if they aren't played with extremely accurate tuning.

## Prime Limit

The simplest, most direct and unambiguous musical intervals are built up from 2:1 ratios (octaves) and 3:2 ratios, as in the previous example (perfect fifths). Tuning systems based on stacking pure octaves and fifths are called *Pythagorean*.

Pythagorean intervals are represented on the keyboard as the row of un-colored "white" keys down the middle. The number printed on the key tells you how it relates to the tonic

( $\frac{1}{1}$  at the center of the keyboard) and you can see for yourself that these ratios only involve factors of 2 and 3.

Even Pythagorean intervals can become unclear, though, if you stack a lot of 2:1 and 3:2 ratios. The Pythagorean major third, for instance, is 81:64. (This interval appears on our keyboard between  $\frac{8}{9}$  and  $\frac{9}{8}$ .) We can get a simpler and more pleasant-sounding major third of 5:4 by allowing for factors of 5 as well. By admitting the use of a larger prime factor, we expand the expressive power of the tuning system.

Often we refer to systems of just intonation by the largest prime number factor they use. Pythagorean could be called 3-limit just intonation. When 5:4 became culturally accepted as a “real” note, that marked the transition to 5-limit just intonation. (5-limit JI is also called “triadic just intonation” because it allows for the basic 4:5:6 major triads and 10:12:15 minor triads that are foundational to modern music. In 12-TET those ratios are only a coarse approximation, but the relationship is still strongly implied.)

On the Mosaichord (assuming it was made in accordance with the standard default color scheme), notes that are related to the “tonic” (the  $\frac{1}{1}$  note in the center) by powers of 5 are either yellow or blue, depending on if you’re multiplying by 5 or dividing, respectively.

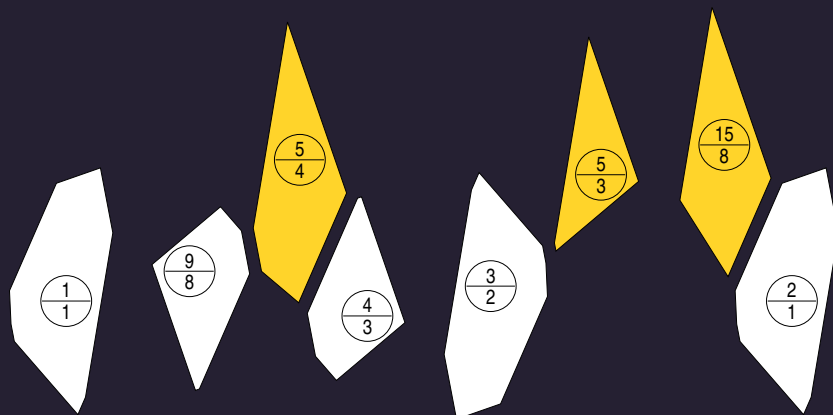
(Keys that share the same color are related to each other by Pythagorean intervals. Thus if you restrict yourself to one color you’re in a Pythagorean scale which, aside from the plain keys, is transposed from  $\frac{1}{1}$  by some non-Pythagorean interval.)

## Scales and Chords

To relate the Mosaichord to a conventional (*Halberstadt*) piano layout with its familiar black and white keys, natural (un-stained) and yellow Mosaichord keys correspond to the white piano keys if we’re in the key of C, with the natural keys being the “neutral” scale degrees like the perfect fourth ( $\frac{4}{3}$ ) and fifth ( $\frac{3}{2}$ ), as well as the Pythagorean major second ( $\frac{9}{8}$ ) and minor seventh ( $\frac{16}{9}$ ), and the yellows being the major scale degrees like the major third ( $\frac{5}{4}$ ), sixth ( $\frac{5}{3}$ ), and seventh ( $\frac{15}{8}$ ). The blue Mosaichord keys are like the black piano keys - minor third ( $\frac{6}{5}$ ), sixth ( $\frac{8}{5}$ ), and a 5-limit variation of the minor seventh ( $\frac{9}{5}$ ).

You may notice some redundancy – the piano has twelve notes per octave, but if you add up all the naturals, blues, and yellows, the keyboard has 16 of those keys per octave. That’s because it’s often necessary when performing in just intonation to make distinctions between pitches that are equivalent in 12-TET.

For instance, consider the just version of the C major scale: C is  $\frac{1}{1}$ , D is  $\frac{9}{8}$ , E is  $\frac{5}{4}$ , F is  $\frac{4}{3}$ , G is  $\frac{3}{2}$ , A is  $\frac{5}{3}$ , B is  $\frac{15}{8}$ , and we return back to C with  $\frac{2}{1}$ . Seven distinct notes if we don’t count the repeated C.

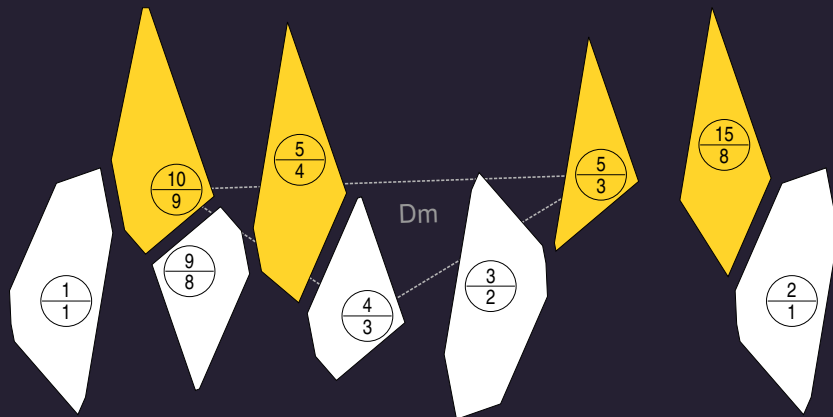


We usually consider the chords in the key of C to be C major, D minor, E minor, F major, G major, A minor, and B diminished. A just major chord, as we've said, is made of sounds in a 4:5:6 mutual resonance and minors are 10:12:15. (Octave inversions create some variations on those, but the mathematical relationship is the same if we ignore powers of 2 – 3:4:5 and 5:6:8 are just inversions of 4:5:6.)

The C major chord is easy, that's just C, E, and G or  $\frac{1}{1}$ ,  $\frac{5}{4}$ , and  $\frac{3}{2}$  which could just as well be written as  $\frac{4}{4}$ ,  $\frac{5}{4}$ , and  $\frac{6}{4}$ . Clearly a 4:5:6 resonance. G major works out nice too. G, B, and D is  $\frac{3}{2}$ ,  $\frac{15}{8}$ , and  $\frac{9}{8}$ . We could re-write that as  $\frac{12}{8}$ ,  $\frac{15}{8}$ , and  $\frac{18}{8}$  (shifting the D up an octave to simply things, since inversions are still the same chord), or  $\frac{3}{2} * (\frac{4}{4}, \frac{5}{4}, \frac{6}{4})$ . The math is a little more complicated, but it works. F is  $\frac{4}{3}, \frac{5}{3}, \frac{2}{1}$  which also works out nicely.

With D minor we encounter a problem. D, F, A is  $\frac{9}{8}, \frac{4}{3}, \frac{5}{3}$ . This does not make the 10:12:15 ratio of a minor chord. Rather, it makes 27:32:40. If we play it anyways, it sounds off. The  $\frac{4}{3}$  and  $\frac{5}{3}$  make a proper  $\frac{5}{4}$  major third between them, but  $\frac{9}{8}$  and  $\frac{4}{3}$  make an interval of 32:27, not the 6:5 just major third interval we want.

We can fix the problem by substituting  $\frac{10}{9}$  for  $\frac{9}{8}$ . It's only a little bit flatter than  $\frac{9}{8}$ . Why not just use  $\frac{10}{9}$  all the time as our "official" D note? Well, then the G major chord wouldn't work out because we used  $\frac{9}{8}$  there. We haven't left the key of C major, but we find that if we want to use a D minor chord, our simple major scale needs at least 8 notes instead of 7 because there are two slightly different Ds that we might use in the same song.

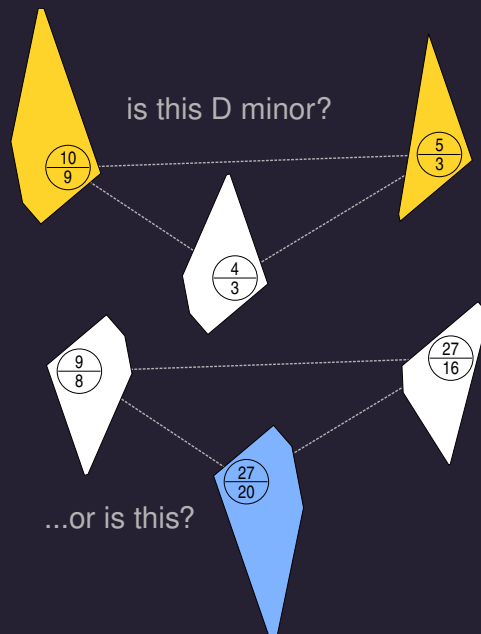


We could also have stuck with  $\frac{9}{8}$  as the root of the d minor chord, but instead introduced  $\frac{27}{20}$  and  $\frac{27}{16}$  as new notes to make the 4:5:6 resonance work. That may work just as well, and the 7-Limit Mosaichord has physical keys for all these notes, so you can try them both and use whichever one sounds best in that context.

(A minor happens to work out nicely using  $\frac{1}{1}$ ,  $\frac{5}{4}$ ,  $\frac{5}{3}$ , and then things get weird again with Bdim, which seems to sound best using  $\frac{15}{8}$ ,  $\frac{9}{8}$ ,  $\frac{21}{16}$ , for a 5:6:7 resonance. That  $\frac{21}{16}$  note isn't just not a note that's commonly regarded as belonging to the major scale, it's a seven-limit interval that doesn't even exist in 5-limit just intonation.)

One challenge of adapting music from 12-tone equal temperament to just intonation is that you have to be aware of these distinctions that didn't exist in 12 tone equal temperament – which can lead to strange situations like chord progressions that are "supposed" to lead back to the starting chord but actually end up slightly pitch-shifted.

This is related to another issue, that in just intonation you can run out of notes if you go too far in any one direction. There are an infinite number of ratios and we can't put them all on a keyboard with a finite number of keys, so the selection of available keys becomes a limitation of each instrument. 12-tone equal temperament is like a round Earth, where you can travel any direction you want, but keep coming back to where you started. Just intonation is more like a flat Earth.



It's potentially much bigger, and has huge swathes of unexplored territory and fascinating features, but eventually if you go too far you can fall off the edge.

The Seven Limit Mosaichord keyboard (named for its prime limit of seven) is meant to strike a balance between having enough notes to play most conventional 5-limit music as long as it doesn't stray too far from the tonic (e.g. by making complicated key changes), while also making accessible some of the largely unused-in-modern-music intervals that include factors of seven.

As for those seven-limit intervals, the purple keys have a frequency that's related to the tonic by a multiple of seven, whereas the green keys have a frequency that's divided by seven. This can be thought of as the prime seven equivalent of the familiar major and minor of five-limit music, in which the majors are multiples of five and the minors are divided by five. That's if we're thinking mathematically. Perceptually it seems to go the other way though:  $\frac{7}{6}$  tends to be perceived as a different kind of minor third and  $\frac{9}{7}$  sound like a different kind of major third.

The seven-limit intervals have a characteristic sound that is seldom heard in mainstream western music outside of barbershop harmony.

The keyboard also has a pair of keys per octave,  $\frac{7}{5}$  and  $\frac{15}{14}$ , which include both a factor of seven and five in the ratio. These are a greenish-yellow and bluish-purple, respectively.

The physical placement of keys is not haphazard, but rather is designed so that musical intervals and chord shapes are consistent anywhere on the keyboard. Transposing a song can be done just by beginning from a different starting point. (That is to say that the keyboard is "isomorphic".) There is a caveat, though, that it's necessary to pick a starting location where you won't run into a situation where some notes that the song requires aren't available on the keyboard because you went too far in one direction and reached the "edge" a situation that doesn't come up on equal-tempered instruments.

One useful way to map out the musical territory of a tuning system is using a kind of grid called a lattice or tonnetz. Octaves are usually ignored in lattice diagrams (by convention, fractions are normalized to lie between  $\frac{1}{2}$ , and  $\frac{3}{2}$  by multiplying or dividing by 2) and scale degrees are laid out in a triangular grid where the horizontal axis usually corresponds to perfect fifths, major thirds are usually along the right-leaning diagonal, and minor thirds are usually on the left-leaning diagonal. Generally, to move up and right involves multiplying by 5, and moving right involves multiplying by three. Moving in the opposite directions is the same but dividing instead of multiplying.

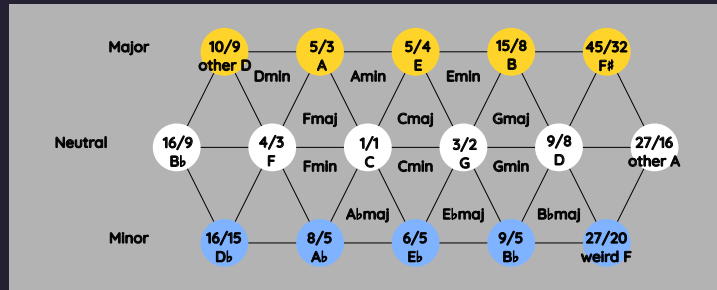


Figure 1:

In this layout, major triads form upward-pointing triangles, and minor triads form downward-pointing triangles. We can also see that there is more than one D, A, and F, and Bb.

Displaying the 7-limit notes is a bit more complicated, as each new prime adds a new axis to our lattice, and our diagram has to flatten everything down to 2 dimensions.

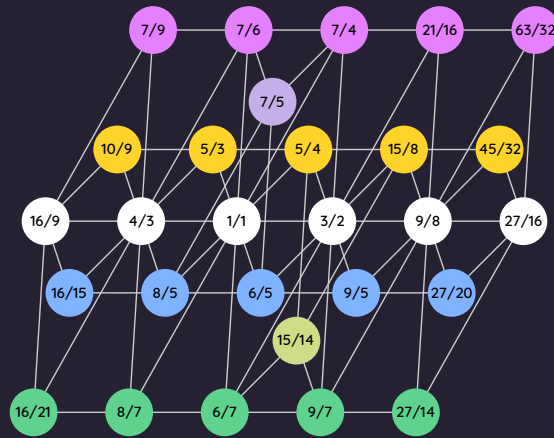


Figure 2:

Here we have the 7-limit intervals added in, including  $\frac{7}{5}$  and  $\frac{15}{14}$ . This lattice can be imagined as a three-dimensional object with the 5-limit intervals coming out of the page/screen at us. The new long, diagonal black lines represent 7:4 and 7:6 intervals.

The layout of the keyboard is actually very similar to this lattice, save that the lattice ignores octaves whereas the keyboard simply has four partially-overlapping copies of these 28 notes, one for each octave.

(For the curious: the locations of the key centers are determined algorithmically – the horizontal position corresponds directly to pitch, while the vertical position is determined by using a carefully-chosen vertical offset for each prime factor in the ratio. The key shapes are

generated by running a Voronoi diagram solver over the previously-calculated key centers. The Voronoi diagram is stretched a bit vertically for better ergonomics and visual appearance.)

That is the general shape of the musical landscape available with this layout; how you use it is up to you.

## Basic Operation

The keyboard receives +5v power through a micro-USB cable plugged into the microcontroller board near the upper right. It does not have an off switch; to turn it off simply disconnect power.

It can function either as a controller instrument for an external synthesizer, or as a stand-alone instrument using the built-in synth engine.

To connect to a laptop as a USB MIDI device, the USB connection is all you need. The keyboard behaves as a class-compliant MIDI device -- though not all MIDI synthesizers have the necessary micro-tuning and expression capabilities to use the keyboard as it was meant to be played. An MPE-capable synth is recommended, but some plain MIDI 1.0 devices may work fine too, usually with a little extra configuration. (See the section on device compatibility for more information.)

(Pictured below is an early version Mosaichord, more recent versions have some minor differences to the housing to protect the microcontroller better and another 3.5mm jack for expression or foot switch input, not yet implemented.)



1. Control knobs – 10 knobs to control synth parameters. Which parameters will depend

on the synth in question.

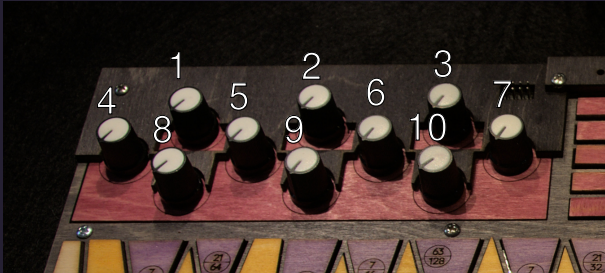
2. Analog input header – used to connect up to four switches or variable resistance devices like expression pedals. (Not currently used.)
3. Status LEDs – 6 programmable RGB LEDs, to be used for status notification, visual cues, or debugging. Current default configuration is for the top left LED glows green when the device is on, and the right-most LED glows when a musical note is playing (more notes is brighter).
4. Menu selection buttons – select one of the menu options displayed on the screen next to it.
5. Screen – displays menu options, navigation, and status. Not a touch screen.
6. Menu navigation buttons – (these are context dependent, and their function will be displayed in the top-right of the screen – top left is usually “back”), and 4-way directional scroll.
7. 3.5mm Stereo audio line out – sends 44.1 khz stereo audio using the built-in synth engine. The audio DAC chip isn’t intended to drive headphones, but in practice it seems to work fine. Some headphones might benefit from the use of a headphone pre-amp. Output impedance is at least  $470\Omega$ .
8. Microcontroller / usb connection – central processing device of the keyboard, a Teensy 4.0 microcontroller. It runs the device firmware and supplies power from the USB port to the rest of the keyboard.
9. MIDI in/out – standard DIN5 MIDI connectors. MIDI-out sends note, pitch, and expression data to an external synthesizer. Note that DIN-5 MIDI output is normally disabled, as it is slower than USB. Device presets for hardware MIDI synthesizers usually enable it, or it can be enabled manually. Currently the main use of MIDI-in is that the Mosaichord rebroadcasts any MIDI CC message it receives on any channel onto all output MIDI channels (or the MPE control channel, as appropriate).
10. CAN bus – a faster alternative to DIN-5 MIDI. Currently unused, but we’ve verified that the hardware is able to send and receive messages. CAN uses two data wires, a "high" and "low" wire. A large number of devices can share the same bus. The keyboard has three high/low pairs, though typically you would only use one or two. CAN bus is usually used in cars rather than to connect musical instruments, but maybe the Mosaichord can start a new trend.

11. CAN termination resistor toggle switch – CAN bus requires a 120-ohm resistor at each end for termination. If the keyboard is connected to one other device or it's the last device in a chain, then turn this on. Otherwise turn it off. (If, like most people, you aren't using the CAN bus at all then it doesn't matter and you can ignore the switch.)
12. Keyboard expansion bus connector – 9 pin 0.5mm pitch FFC ribbon cable connector for extending the keybed. The electrical design of the keybed makes it at least theoretically possible to daisy-chain multiple keybeds. We haven't tried it yet or even added the necessary support logic to the Mosaichord firmware, but it's an option for future products or ambitious DIY music controller projects.
13. Keys – 113 pressure-sensitive keyboard keys, spanning 4 octaves. The fraction on each key corresponds to the frequency of that note relative to the tonic. For example, if  $\frac{1}{1}$  were configured to 100 hz, then  $\frac{3}{2}$  would be one and a half times that, or 150 hz. Colors correspond to the prime factors 5 and 7 that occur in the top or bottom of the fractions.
14. Octave transpose buttons – transpose base pitch up and down by octaves.
15. Pitch bend bar – an extra long control surface for pitch bend. Unlike the other keys and buttons which act as single pressure sensors, this one has two pressure sensors, an upper and a lower one. Horizontal position doesn't matter, but it can sense up-and-down variations in pressure. It is designed very much like the space-bar on a typewriter-style computer keyboard to be within easy reach of one or both thumbs at any time.
16. Semitone transpose buttons – transpose base pitch up and down by standard 100-cent 12-tone equal tempered semitones. As with the octave transpose buttons, the pitch of the tonic  $\frac{1}{1}$  key in the center of the keyboard is displayed on the screen. (We may add some additional options later, such as transposing by 41-EDO semitones.)

Not pictured: more recent versions have a second 3.5mm jack next to the stereo line out for an expression pedal or footswitch. Firmware support hasn't been added yet.

## Knobs

The ten knobs on the left control various sound parameters parameters.



The default assignment of knobs to functions may well change over time, but at the time of writing this, the standard configuration is this:

1. Volume attack slew limit: sets a limit on how fast a physical key pressure increase translates into an audible sound change. Increase this to slow the attack.
2. Volume release slew limit: sets a limit on how fast a physical key pressure decrease translates into an audible sound change. Increase this if notes are cutting out too quickly.
3. Pressure curve: affects how key pressure correlates to MIDI values across the pressure range. Adjust according to preference. (Low values tend to be more sensitive to light pressure.)
4. Modulation: equivalent to mod wheel found on most MIDI keyboards. Sends MIDI CC 1, affecting all voices. (What exactly this controls varies by synth, but it's most often some kind of vibrato.)
5. Filter attack slew limit: used when we want pressure to affect filter and volume, but we want a different slew rate for each. (Currently only used by the Befaco Midi Thing V2 device preset and the internal synth engine.)
6. Filter release slew limit: likewise for filter release.
7. Reverb: Sends MIDI CC 91, affecting all voices. (Note: some synthesizers don't support reverb or don't respond to that particular CC.)

8. Timbre: sends MPE timbre message, affecting all voices equally. MPE Timbre is defined as MIDI CC 74, which on non-MPE synths is usually filter cutoff. (Not all MPE patch presets respond to timbre, and not all MIDI synths respond to filter cutoff.)
9. Resonance: sends MIDI CC 71, which usually affects filter resonance.
10. Volume: sends either MIDI CC 7 (volume) or 11 (expression), which do effectively the same thing, but many synths support both so that we can modulate individual voice volume using key pressure, and the other can be tied to this knob so we can adjust overall mix volume. By default it's probably best to leave this all the way up.

In general, for the knobs that control MIDI CC parameters, the Mosaichord won't actually send any CC messages until you physically turn one of the knobs. Not all synthesizers respond to knob settings in the same way, and some settings they might ignore entirely. (Surge XT for instance won't generally respond to MIDI CC messages unless you explicitly configure it to.)

The internal synth engine responds to all knobs.

## Menus

The menu system is navigated using the five buttons to the left of the screen and the four additional buttons and direction pad to the left of the screen. The screen displays what the buttons do. The top-left button on the right side navigates up a level. If there are more than five options on the left, ellipses will be shown at the top or bottom to indicate that you can scroll up or down using the direction pad.

Like the knob assignments, the layout of the menus is likely to change substantially over time.

- SETTINGS
  - \* MIDI (settings controlling MIDI / MPE output and external synthesizer settings)
    - USB MIDI (toggle): enables MIDI output over USB
    - DIN5 MIDI (toggle): enables MIDI output over DIN5 (best to leave it off when using USB, otherwise all MIDI messages are rate-limited by slow serial speeds)
    - DEV PRESETS (more submenus): presets to configure Mosaichord to work with various hardware and software synthesizers
    - MPE INIT (action): re-send MPE handshake

- CHANNELS (integer value, 1-16): set number of MPE or MIDI channels
- BEND RANGE (submenu): set to pitch bend range of the synthesizer you're using (usually this is set automatically by the device preset); incorrect value results in incorrect tuning
- PRESSURE CC: which MIDI CC to use for key pressure (usually set automatically by device preset, but you may occasionally want something different)
- MAX P: (integer value, 0-127): maximum key pressure value to send as MPE key pressure or polyphonic aftertouch (usually 127)
- MIN VEL: (integer value, 1-127): minimum MIDI note-on velocity to send; turn this up if like key presses are too quiet
- CC THRU: (toggle) when enabled, all CC messages received by the MIDI-IN port will be multicast on all channels to the MIDI-OUT port.
- \* CONTROLS (what kind of pressure and velocity messages to send)
  - VELOCITY (toggle): send dynamic note-on velocity (when disabled, velocity defaults to 95)
  - PRESSURE (toggle): send dynamic key pressure using MPE key pressure messages or per voice MIDI CC messages
  - POLY AT (toggle): send dynamic key pressure messaging using polyphonic aftertouch
  - P BACKOFF (integer value, no restrictions): microseconds between key pressure messages; turn this up if key pressure response seems sluggish
- \* INTERFACE (screen settings)
  - BRIGHTNESS (submenu): set screen brightness
  - VISUALIZER (toggle): enable visualization of values read by analog-to-digital converters (ADCs) – for debugging mostly, but can sometimes be useful to see when key pressure has maxed out.
- \* SYNTH (internal synthesizer settings that don't already have dedicated knobs)
  - oscillator (submenu): set oscillator among standard waveforms. "Skew" wave is like a triangle wave that's "leaning" to one side so that it's half-way to being a sawtooth.
  - reverb (plate reverb settings)
    - SIZE: room size
    - HIGH DAMP: high frequency damping
    - LOW DAMP: low frequency damping

- LOW PASS: low pass filter on reverb output
  - DIFFUSION: lower values increases echo
- \* DEBUG (settings affecting serial debugger output, view this using Arduino IDE)
  - SHOW RES (toggle): print a table of measured resistance values to serial console
  - SHOW CAL (toggle): print autocalibration information to serial console
  - NOTE-ON 1ST: when micro-tuning MIDI, send note-on before pitch bend
  - UPBEND ONLY: when micro-tuning MIDI with pitch bend, only bend up from a lower note
  - DNBEND ONLY: when micro-tuning MIDI with pitch bend, only bend down from a higher note
  - LOCK: useful for public demos, locks out the control knobs and transpose settings other than (limited) octave transpose
- PATCHES (used to load sound preset patches on an external MIDI synthesizer)
  - \* BANK MSB (integer value, 0-127): most significant bits of MIDI preset bank; range may be restricted by device preset
  - \* BANK LSB (integer value, 0-127): least significant bits of MIDI preset bank; range may be restricted by device preset
  - \* PATCH (integer value, 0-127): program change number to send
  - \* UNLOCK (toggle): turn off any range restrictions in place on MSB and LSB
- NOTES OFF (action): send a MIDI note-off channel on every note number and every channel

For convenience, the PATCHES menu has a couple extra toggles on the right for turning dynamic velocity and key pressure on and off without having to navigate to SETTINGS→CONTROLS.

## Using the Internal Synthesizer

The Mosaichord has its own internal synth engine. No need to enable it; it runs by default. It outputs a stereo signal via the 3.5mm jack on the back of the Mosaichord labeled “line out”.

The internal synthesizer follows a fairly typical subtractive synthesis architecture. Each voice is a basic waveform into a resonant low-pass filter. The amplitude of the oscillator and the cutoff frequency of the filter are both modulated by key pressure.

The internal synthesizer does not currently use note-on velocity.

Most of the adjustable synthesizer parameters can be adjusted using the knobs, but there are a few settings (oscillator type and plate reverb fine tuning parameters) that can be adjusted in the menus (under SETTINGS → SYNTH).

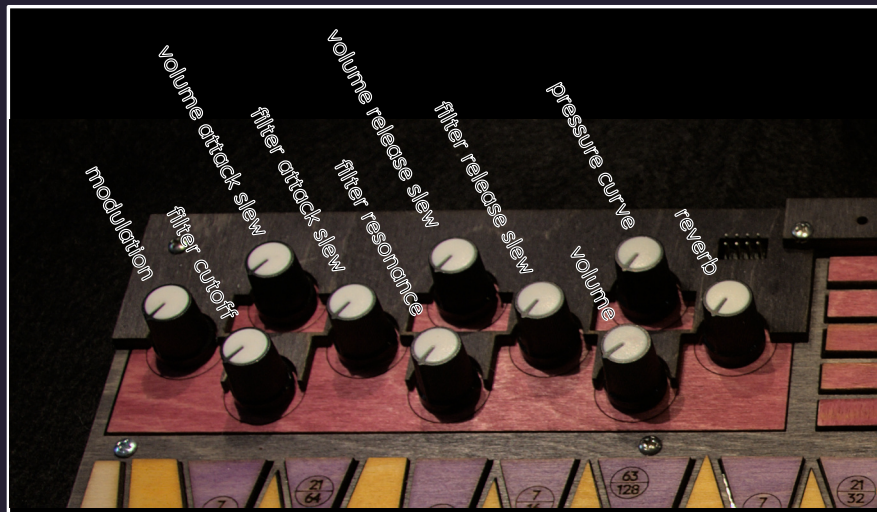


Figure 3:

Some of the knobs take effect immediately, but some synth parameters do not take effect until you turn the knob from its original setting. (Among these are reverb, volume, filter cutoff, and resonance.)

The Synthesizer was created using the Teensy Audio Library. We use an open source stereo plate reverb model from Piotr Zapart.

The internal synthesizer does not currently respond to MIDI messages from the MIDI-in port or USB.

Currently the modulation wheel does a 2hz sine wave volume modulation at 4hz.

The internal synthesizer is a newly-developed feature, over time we expect to add features and expand the range of configurable options.

# Using an external synthesizer

For most electronic musical instruments made in the last approximately 40 years, interoperability between keyboards and synthesizers is pretty straightforward – just plug a DIN5 cable in to the MIDI-out port on the controller and the MIDI-in port of the synthesizer, set them both up to the same channel, and it just works. Or if it's a software synthesizer, then connecting a controller or synth to a laptop with a USB cable is much the same.

We'd like to say the process of setting up the Mosaichord with any MIDI synthesizer was that simple, but as the keyboard makes use of continuous key pressure and requires some very specific tuning capabilities, there are a great many otherwise amazing synthesizers that simply aren't compatible, and the ones that are may require some special attention to configuration details.

We've provided presets for a handful of synthesizers we've tested with the Mosaichord. It would be a monumental task to try to have presets for every MIDI synth ever made, but we hope that the presets we do have will be useful to the people who already own one or two of these synths, and provide a good starting point to add more as time allows.

The pre-defined “device presets” for various synthesizers can be found in the Mosaichord menus under `SETTINGS → MIDI → DEV PRESETS`.

## MPE Synths

Fortunately, the capabilities we want have been standardized in recent years as an extension to the MIDI protocol called MPE. Using the keyboard with MPE-capable synths is the easiest option.

MPE controllers are expected to send a handshake when they connect, to tell the synthesizers that it's using MPE and how many channels to use. The keyboard sends this when booting up or when switching to an MPE-enabled preset, but it's also possible to send it explicitly by invoking `SETTINGS → MIDI → MPE INIT`.

## Surge XT

Surge XT is probably the easiest option to get up and running quickly. It is a free and open source MPE-capable software synthesizer that runs under Windows, MacOS, and Linux.

A few things are worth paying attention to. Enable MPE by clicking on the MPE button. The MPE pitch bend range should be set to 48 semitones. (This is in the menus and is separate

from the “regular” bend depth setting near the upper left of the main window.)

Better tuning precision can be achieved with a narrower MPE pitch bend range so feel free to change it, just be aware that the keyboard and Surge have to agree on what the range is or the sounds that come out will be out of tune.

Most of the regular patches aren’t designed to react to MPE key pressure, but there are a handful of MPE-specific patches including some contributed by Roger Linn for the Linnstrument. (The Mosaichord keys lack the per-note “Y axis” control that the Linnstrument uses to affect MPE Timbre, though it’s possible to adjust the timbre for all the notes at once using knob 8.)

By default, Surge XT ignores most of the standard MIDI CCs unless those have been explicitly enabled and bound to some parameter. For instance, to use knob 7 as a reverb control, you can bind one of the macros to MIDI CC 91 (or use “MIDI learn” and twist the knob), then have macro 8 modulate the reverb send level on your current patch.

The non-MPE patches can often be adapted for more expressive control by setting MPE PRESSURE as a modulation input for Amp Gain, the cutoff for filter 1 or filter 2, or any number of other controls.

When connecting the keyboard over USB, depending on operating system you will likely have to select SEVEN LIMIT MOSAICHORD under audio/midi settings in the OPTIONS menu in the top left.

We have provided a Surge XT device preset in the menu under SETTINGS → MIDI → DEV PRESETS → SURGE XT, but it’s also the default preset on boot up.

(This preset assumes the connection will be over USB. You can use DIN5 instead by manually enabling SETTINGS → MIDI → DIN5 MIDI.)

This device preset may work with other MPE-capable synths, but we haven’t tested.

## Multitimbral MIDI Synths

Many synthesizers that don’t support MPE can still be used with the Mosaichord, but there are a number of things that can go wrong if the synthesizer and the Mosaichord are not configured correctly, and even when they are they may require some trial and error to find what works best.

The most common thing to go wrong is that the tuning is off. To understand how this happens, it helps to know how the Mosaichord is able to use arbitrary tunings on synths that, for the most part, were never designed to support it in the first place.

Pre-MPE MIDI 1.0, originally standardized in 1983 and used ever since, doesn't have any explicit support for tuning. (Well, technically it does. There's MTS, the MIDI Tuning Standard ratified in 1992, but it's not widely supported.)

What MIDI 1.0 *does* have that is supported by almost all MIDI synths is pitch bend. It even has a native resolution of 14 bits, which is enough for pretty accurate tuning even if the pitch bend range is very wide. The main difficulty with pitch bend, though, is that it's not applied on a per-note basis. Rather, it affects *every* note that's currently being played in the current channel. That's not very useful if we want to play a chord and have every note perfectly in-tune (according to a definition of "in tune" that differs substantially from 12-tone equal temperament). We want to be able to bend every note individually.

It's possible to have per-note pitch bend, though, if we constrain ourselves by only playing one note at a time per MIDI channel. MIDI has 16 channels, so that gives us 16 voices of polyphony, which is pretty reasonable. As a side benefit of using multiple channels, we can also use other MIDI CC ("Continuous Controller") messages that affect the whole channel on a per-note basis, like volume or filter cutoff.

Not all synths support multiple channels. The ones that do are called "multi-timbral" as the feature allows the synth to produce several different timbres at the same time, as if it were multiple separate instruments. Most analog polyphonic synths are unfortunately monotimbral, but digital synths that use ROM-based sample presets (often called "romplers") were often multi-timbral.

Spreading the notes across multiple channels and using pitch bend to get the exact notes we want has drawbacks, though: the Mosaichord controller must send MIDI commands to the synth without knowing if they're being interpreted correctly, and that interpretation depends on settings like the pitch bend range. If it thinks it's something different than what it is, the notes will be either jarringly out of tune if it bends too far, or somewhere between correct tuning and 12-EDO if it doesn't bend far enough. We might even encounter situations where a synth is configured with a different pitch bend range on different channels.

The pitch bend range defaults vary by synth, or even between preset patches on the same synth. Two semitones up and down is a common default. Fortunately there is a standard MIDI RPN to set a channel's pitch bend range explicitly, and most synths we've tried support it.

Besides having to be careful about pitch bend range configuration, using multiple channels creates other inconveniences. If we want to change a patch or some sound engine setting, we have to make the exact same setting change to all 16 channels. We *could* do this manually, but it would be tedious, so instead we integrate much of that functionality into the Mosaichord, so you can make the change once and it propagates that change across all the

channels automatically.

The most common use of this is changing the active patch preset. Most synths have banks of presets arranged into convenient categories like “keyboards”, “strings”, “brass” and so on. At the level of MIDI messages, though, we deal in terms of “banks” which can each have up to 128 patches selected with “program change” messages, and we can select what bank is active by sending a 14-bit number split into 7 “most significant” MSB bits and 7 “least significant” LSB bits (set using MIDI CC messages). That gives 16384 possible banks. Most rompler synths of the 1990’s and 2000’s have maybe somewhere around half a dozen or so, and the rest are empty space. Where the real banks are in that large address space varies greatly by manufacturer and model, but it’s usually listed somewhere in the user manual.

The device presets we’ve defined usually impose an upper and lower bound on the MSB and LSB to make it easier to find the populated banks.

MSB, LSB, and patch preset (i.e. Program Change) numbers can be found in the Mosaichord menu under PATCHES. Select the value you want to change, and press the right and left direction pads to increase or decrease the value. Changes are sent immediately. You can also “unlock” the MSB and LSB range limits imposed by the device preset.

Note: the Mosaichord uses 0-based counting (values 0-127), whereas some manufacturers prefer 1-based counting (1-128).

## E-mu Proteus 2000

The Proteus 2000 (released in 1999) is a great synth to use for microtonal music, as it allows user-defined tuning tables. We don’t use that feature though, because the one-note-per-channel pitch bend trick accomplishes the same thing. (Maybe some day.)

To use the Proteus 2000, set its MIDI mode to “multi mode” if it isn’t already (using the main menu on the Proteus 2000), connect the MIDI-out on the Mosaichord to MIDI-in port A on the Proteus 2000, and select the device preset under SETTINGS→MIDI→DEV PRESETS→E-MU→PROTEUS 2K on the Mosaichord.

Using pressure to modulate volume seems to work pretty well on the Proteus 2000 with most patch presets, so we leave it enabled by default in our “Proteus 2k” device preset. In particular, this synth applies reverb *after* channel volume rather than before, which is nice as it means the reverb tails aren’t unnaturally affected by key pressure. (When using velocity and key pressure at the same time, it may be helpful to raise the minimum velocity and pressure values under SETTINGS→MIDI→MIN VEL and MIN P.)

If you’d like to use the Proteus 2000 knobs to edit device presets and would like them to affect all MIDI channels at the same time, then set “knobs midi out” in the Proteus 2000

MIDI configuration to “transmit”, run a second MIDI cable from the MIDI-out A on the Proteus back to the MIDI-in on the Mosaichord, and enable `SETTINGS→MIDI→CC THRU` on the Mosaichord.

Turning knobs on the Proteus 2000 will cause it to send MIDI CC messages corresponding to the values of the knob being turned. The Mosaichord will read these CC messages and broadcast them on all channels.

The Proteus 2000 has a feature called a “Multisetup” (accessed via the “multi” button on the front panel) that allows for naming and saving the configuration of all 16 channels into one of 128 preset slots that can be loaded later, either from the Multisetup menu on the Proteus 2000 or by loading the preset number from Mosaichord `PATCHES` menu after first unlocking the MSB/LSB range and loading the Multisetup bank using `MSB = 80` and `LSB = 0`.

## **Korg Trinity**

The Trinity (released 1995) was available with a keyboard, or as a rack unit called the TR Rack. Connect the MIDI-out on the Mosaichord to the MIDI-in on the Trinity.

To use all 16 MIDI channels, press the `GLOBAL/MULTI` button a couple times to enable “Multi” mode and select the Korg Trinity device preset on the Mosaichord under `SETTINGS→MIDI→DEV PRESETS→KORG→TRINITY`

Using pressure for key volume doesn’t seem to produce a good result on most preset patches, so we leave it off by default.

## **Korg X50**

The X50 (released in 2007) is a less expensive version of the Korg Triton. The X50 device preset may work for the Triton as well, but we have not tested it. To use the X50, set it to “Multi” mode by pressing the `MULTI` button, connect the MIDI-out on the Mosaichord to the MIDI-in port on the X50 with a DIN5 MIDI cable, and select the X50 device preset on the Mosaichord under `SETTINGS→MIDI→DEV PRESETS→KORG→TRINITY`.

## **Korg SP-300**

The SP-300 is a multi-timbral stage piano released in 1995. Connect the MIDI-out on the Mosaichord to the MIDI-in on the SP-300 and select the device preset under `SETTINGS→MIDI→DEV PRESETS→KORG→SP-300`. That preset enables both velocity and key pressure by default, but most presets work best with just one or the other.

An interesting thing about the SP-300 is that the internal circuitry isn't actually very big. It's possible to remove the electronic internals and speakers from its bulky enclosure and disconnect the 88-key keybed, and transfer it to a much smaller box. Then it becomes a portable desktop MIDI sound module. It also functions as a powered speaker, as the SP-300 has audio line-in jacks.

The SP-300 has only a single bank with 30 sounds, so there's no need to change MSB or LSB.

## **Roland XV-2020**

The XV-2020 is a multi-timbral (mostly) sample-based MIDI synth released in 2002, as a continuation of their JV/XV product line.

To connect to the Mosaichord, connect a MIDI cable from the MIDI-out of the keyboard to the MIDI-in of the XV-2020, turn the XV-2020 on, and press the "value" button until there is a single red light by the "GM" label. That puts it into "General MIDI" mode, but also sets it to receive on all channels. General MIDI is a bank of standard instruments, so that, for instance, patch 0 is always a piano, patch 40 is always a violin, and so forth. (Here, we're counting from 0-127, but often devices and documentation will count from 1-128. We don't have to use the General MIDI sounds though, we can select a different bank by changing the bank MSB ("most significant bits") and LSB ("least significant bits").

Once the XV-2020 is up and running in GM mode, select the XV-2020 preset on the Mosaichord via `SETTINGS→MIDI→DEV PRESETS→ROLAND→XV-2020`.

By default, we set the MSB to 87 and allow LSB to vary from 64-67. This accesses the internal preset banks. To access the user presets and any expansion cards will require unlocking the MSB/LSB range. Select `PATCHES→UNLOCK` in the menu.

We have found that modulating channel volume according to key pressure on the XV-2020 doesn't usually create a satisfactory result (using both key pressure and velocity for volume at the same time generally don't go well together, and the XV-2020 has a tendency to lag when it receives too many MIDI CCs in quick succession), so we turn it off by default on this preset.

The XV-2020 supports polyphonic aftertouch, as an alternative way to do receive per-note key pressure data. This is disabled by default on the Mosaichord, but you can enable under `SETTINGS→CONTROLS→POLY AT`.

## Roland RD-300NX

The RD-300NX is a multi-timbral stage piano, released in 2011. Connect the MIDI-out on the Mosaichord to the MIDI-in on the RD-300NX and select the device preset under SETTINGS→MIDI→DEV PRESETS→ROLAND→RD-300NX. That preset enables both velocity and key pressure by default, but most presets work best with just one or the other.

It seems all the patch presets on the RD-300NX use MSB 87 or 121, and most use an LSB from 0 to 4, with a few outliers that use 67, 69, and 72. Consult the RD-300NX Patch/Performance List document from Roland for the complete list. We enable both key pressure and velocity by default, but as usual, you'll probably get a better result by only enabling one or the other.

## Yamaha FB-01

The FB-01 works surprisingly well with the Mosaichord, considering it was released in 1986. Connect the MIDI-out of the Mosaichord to MIDI-in on the FB-01, and select the device preset under SETTINGS→MIDI→DEV PRESETS→YAMAHA→FB-01.

The device preset is configured to do some unusual things that are specific to the FB-01. The FB-01 has 8 voice circuits, and these are statically assigned to MIDI channels. Usually, a user would assign them all to a single channel, but in our case we want to assign a single voice to each of the first 8 channels. We do this by sending some MIDI SysEx commands to set each MIDI channel and “note count”. If this doesn't work, you may have to manually configure the FB-01 from the front panel. (One reason why this might not work is that the FB-01 system channel is set to something other than 1. “System channel” is something specific to the FB-01 and is independent of MIDI channels.)

The FB-01 does not use the same MIDI CC MSB/LSB system for loading banks of preset patches that became standard on later MIDI hardware; instead it uses a special bank select SysEx message. We've set the FB-01 device preset so that it sends that other SysEx message, and changing the bank LSB as normal on the Mosaichord sends that other SysEx message instead.

Also, when changing patches we set the pitch bend range explicitly each time (as the pitch bend range otherwise varies by patch), and we explicitly set “detune” to 0 and disable the LFO, as it usually detracts from the sound more than it helps when working in just intonation. (Most patches have some vibrato that you can add back in using the far left knob 4 on the Mosaichord, which acts as a mod wheel.)

## Yamaha MOX8

Released 2011, the MOX8 works with the Mosaichord with a bit of setup. First, set it to receive on all channels by pressing the “SONG” button. Connect the MIDI-out on the Mosaichord to MIDI in on the MOX8, and select the MOX device preset under SETTINGS→MIDI→DEV PRESETS→YAMAHA→MOX8.

On the MOX8, you can press the “MIXING” button then F2 to see what patch is active on each channel. These should all be the same.

Depending on what preset was previously active, you may find, though, that the channels don’t all *sound* the same. They might all have different values for filter cutoff, sustain, reverb send level, or any of the other myriad sound settings.

These settings can be checked manually and set by pressing each “PART” button over on the right of the MOX8 control panel and then selecting/adjusting each control over on the left side of the MOX8 control panel. For convenience, we use MIDI CCs to set *most* of these to reasonable defaults when the MOX8 device preset is selected.

We also recommend turning insertion effects off. It appears these can only be enabled on three channels at a time, which causes distracting sound changes when only three of every 16 notes have them enabled.

## MIDI → CV devices

Sometimes we might want to use the keyboard with synthesizers that work based off of analog control voltage signals and gate inputs rather than MIDI. For that, there are MIDI to CV devices that will do the necessary conversion for us.

### Arturia Keystep Pro

The Keystep Pro (or KSP) in addition to its many other uses can serve as a MIDI to CV converter, to interface with a modular synthesizer. The KSP does not, as of the time of writing this, support MPE, but it does allow us to use a separate channel for each voice, which is good enough for our purposes.

Used as a MIDI to CV converter, KSP supports up to four channels of polyphony. Each channel has its own dedicated pitch CV, modulation CV, and gate output. The pitch CV is affected by MIDI pitch bend.

A conventional setup would be to run the four pitch CV outputs into four identical voltage-controlled oscillators (VCOs), the modulation CV inputs would be connected to four iden-

tical voltage-controlled amplifiers (VCAs) controlling the volume of the four VCOs, and the gates would trigger envelope generators to further modulate volume and/or filter cutoff.

Getting a good result will depend a great deal on how accurately the VCOs track pitch CV and how well they stay in tune. I've had decent luck with 3340-based VCO designs, but they require careful calibration for best results.

The KSP settings can be configured from a computer using Arturia's MIDI Control Center. (Some of the settings may be configured on the device itself, but there are some important ones that can't as of the time this is written.)

You'll want to configure the KSP's four voices to be on MIDI channels 1, 2, 3, and 4 with a pitch bend range of  $\pm 2$  semitones. The CV output for all channels should be set to 1v/octave (unless you're using a modular system based on a different standard) and the Modulation CV out for all four channels should be set to "mod wheel" and should all have the same output voltage range.

From there, connect a MIDI cable from the MIDI-out port of the Mosaichord keyboard to the MIDI-in port of the KSP, select the SETTINGS→MIDI→DEV PRESETS→ARTURIA→KEYSTEP PRO device preset using the Mosaichord menu, tune your oscillators as needed, and you should be good to go.

The internal synthesizer on the Mosaichord can be used as a reference pitch to tune your oscillators against.

If you run into tuning problems, be sure to check that the KSP firmware is up-to-date. As of writing this, version 2.5.20 is the latest.

## Befaco MIDI Thing V2

The MIDI Thing V2 (or MTv2) is a 12-channel MIDI to CV converter that supports DIN5 connection or direct USB connection. (It can act as USB host.) However, we recommend connecting the Mosaichord using DIN5 MIDI, as the MTv2 does not recognize the Mosaichord as a USB device. This is a known bug in the Teensy USB implementation, that Teensy-based USB hosts such as the MTv2 don't always recognize Teensy-based USB peripherals, which the Mosaichord is. This may get fixed eventually, but for now using the MIDI cable is a workaround.

The MTv2 does not directly support MPE, but we can get effectively the same result just by configuring each voice to use its own MIDI channel.

The Mosaichord's MTv2 device preset uses the 12 outputs of the MTv2 in an unusual way. We assume 4-voice polyphony, with each voice on a separate row. The first column is pitch, second column is volume, and third is filter cutoff.

A separate gate output is not strictly necessary when key pressure is used to control volume via a voltage-controlled amplifier. If zero key pressure results in zero volume, the note is effectively off. This frees up an extra channel per voice to use for something else, and so we use that to have separate control over volume and filter cutoff. These are both affected by key pressure, but by applying different slew limits (using the attack/release control knobs for filter and cutoff), they can diverge.

A lot of other configurations are possible; we went with this four-voice setup because it's relatively straightforward to get a 4-voice polyphonic setup using Doepfer's polyphonic modules (i.e. the A-132-8 polyphonic VCA, A-105-4 polyphonic VCF, and A-111-4 polyphonic VCO) and it doesn't require a polyphonic envelope generator like the A-141-4.

Something else to be aware of is that the MTv2 doesn't (as of the time this was written) have any smoothing on CC or MIDI channel pressure outputs, which causes some audible "stair-stepping" artifacts that sound like white noise, especially during quick attacks. It's most noticeable with sine waves, not so much when the oscillators have a lot of harmonic content like sawtooth waves, or when there's an aggressive low-pass filter setting.

You can configure the MTv2 directly from the device itself using its built-in screen and knob. You will need to add four "voices" (V1, V2, V3, and V4). You can create each new voice as "only note", then "add to voice" channel pressure then MIDI CC 74 (filter cutoff). Set pitch bend range to 2 semitones. I use a -5 to 5 volt range for pitch and 0 to 10 volts for key pressure and filter cutoff, but this may depend on what modules you're outputting to. Set the four voices to MIDI channels 1, 2, 3, and 4 respectively.

Fortunately, you only need to do this once, then long-press the encoder button to bring up the global menu and use "save mem" to save the configuration. By default, it boots to the last configuration saved.

Connect the Mosaichord via DIN5 MIDI using the DIN5-to-3.5mm adapter that came with the MTv2,

## Untested

- Mutable Instruments Yarns
- Expert Sleepers FH-2
- Ornament and Crime Teensy 4.1 version

## Unsupported

- Korg SQ-64 (no support for pass-through of pitch bend when used as MIDI to CV converter)

## Mono Synths

Monophonic synths are, in most respects, easier to support than polyphonic synths, as we only need to manage a single channel. We can generally just use the front panel controls normally.

Monosynths tend to have some sort of note-priority logic – usually either the highest note, or the lowest note, or the most recent note is the one that’s actually played.

Since we need to know which note is currently being played in order to send the right pitch bend command, we make sure to only have one active note at a time, by sending a note-off before the next note-on. We use most-recent-note priority.

### Moog Slim Phatty

The Slim Phatty (released in 2011) is pretty straightforward to use. Unlike most analog mono synths, it provides MIDI control over most of the synth parameters.

One thing to be careful of is that preset patches can override the pitch bend range. We send a MIDI RPN to reset pitch bend range when selecting a preset patch from the Mosaichord menu system, but if you change presets from the front panel of the Moog, the pitch bend range could get off from what the Mosaichord thinks it is. You can fix this by either selecting presets from the Mosaichord, or by resetting the pitch bend range (SETTINGS→MIDI→BEND RANGE).

The Slim Phatty responds to MIDI CC 11 (channel volume) but it produced some audible artifacts when I tried it, so we default to using CC 19 for pressure, which affects filter cutoff.

## Tuning Table Synths

Some mono-timbral synths can be used with the Mosaichord by loading a custom tuning table. The details of how exactly this is done vary a great deal between different brands and models of synthesizer.

There is a tuning table standard that was eventually added to the MIDI standard called MTS, but we don't currently support it.

(Some multi-timbral synths that support custom tuning tables, such as the Yamaha FB-01 and the Emu Proteus 2000, already work with the Mosaichord by using multiple channels and pitch bend.)

## **Yamaha DX7 with Grey Matter Response E! expansion board**

This one is a bit unusual. The original DX7 did not have custom tuning table support (though it was available in the later DX7IID and DX7IIFD). However, Grey Matter Response developed a popular add-on board for the DX7 that essentially replaces the original firmware with a custom firmware and adds additional patch storage memory. Among the new features supported by the E! firmware is custom tuning table support.

To use the Mosaichord with a DX7 with the E! expansion, connect the MIDI-out of the Mosaichord to MIDI-in of the DX7, and turn them both on. By default, the E! firmware blocks incoming SysEx messages, so you'll have to turn off the midi SysEx filter. (Hit "function" then "operator select" several times until it says "keyboard control" then hit the "2" button and use the value slider to turn Sys/Ex on.) Then load the E! device preset by selecting SETTINGS→MIDI→DEV PRESETS→YAMAHA→DX7 WITH E! in the Mosaichord menus.

If the tuning table is loaded correctly, the keyboard will sound the notes with their proper just intervals. If for some reason the tuning table doesn't load, the pitch range will be very widely stretched, as the 113 keys will span over 9 octaves, with each key's pitch separated by an equal tempered semitone.

The DX7 will be able to respond to note-on velocity, but not key pressure. (We would use MIDI 1.0 polyphonic key pressure messages for this, but unfortunately the E! firmware does not respond to those.) For more expressive playing, we'd recommend the Yamaha FB-01 – despite its more primitive sound engine (4-operator fm versus 6-operator in the DX7), we can modulate the volume of individual notes by spreading them across 8 MIDI channels and using channel volume CC messages.

Semitone and octave transpose features work as usual with the DX7 with E! device preset, though be aware that the Mosaichord sends a 521 byte SysEx message to update the tuning table whenever the transpose setting changes, so it may introduce some slight latency.

The E! tuning table format is different from the one used by the DX7IID and FD, so this device preset will not work with those synths.

## Dexed

Dexed is an open source re-implimentation of the DX7 sound engine, and can be used to play DX7 patches on a computer. Dexed supports MPE, but we were unable to get it to recognize pitch bend commands. However, it does support custom tuning tables via Scala scale files (\*.scl) and keyboard maps (\*.kbm). If we load the appropriate tuning table, Dexed can work in pretty much the same way we use the DX7 with the E! expansion.

To use Dexed with the Mosaichord, connect it to your computer with a usb cable, run Dexed, and select the Mosaichord midi input device (however that is done on the operating system you're using).

On the Mosaichord, select the SETTINGS→MIDI→DEV PRESETS→DEXED device preset from the menu. Within Dexed, click the PARM button and input the *seven-limit-mosaichord.scl* and *seven-limit-mosaichord.kbm* files available from the “extras” subdirectory in our source code repo here:

<https://github.com/jimsnow/microtonal-controller/tree/main/extras>

Enable the transpose “12 as SCL” option, otherwise some patches are likely to sound quite strange. Leave the MPE setting off<sup>1</sup>.

The transpose buttons on the mosaichord will not work with the DEXED device preset, but Dexed has a transpose option in the interface. You can also edit the .kbl file as desired.

---

<sup>1</sup>You can always try out MPE if you really want to to see if you get better luck than we did, but in that case don't load the .scl or .kbl file and stick with the SURGE XT device preset instead of the DEXED device preset.

# Firmware updates and modification

While we hope that the Mosaichord works just fine for most people in its stock form, this is a new product and we expect to be continually adding software features over time. We have also released the firmware as an open source project to enable our users to make changes to suit their preferences as well, if they so desire, as this is ultimately *your* instrument not ours.

It is also not possible for us to develop and test device presets for every kind of synthesizer, and we expect there will always be some obscure configuration options that aren't available in the menus and can only be accessed by modifying the program directly.

For all these reasons it may, from time to time, be necessary to update the the device firmware.

## Install/Setup development tools

The Mosaichord uses the Teensy 4.0 microcontroller, and firmware development is normally done using the Arduino IDE.

Arduino can be downloaded from <https://www.arduino.cc/en/software>. (Under linux, you may be asked to tweak some serial group permissions.)

Support for the Teensy microcontroller can be added from within the board manager of the Arduino IDE, once you've added a URL to the board manager settings.

Under FILE→PREFERENCES→SETTINGS, add [https://www.pjrc.com/teensy/package\\_teensy\\_index.json](https://www.pjrc.com/teensy/package_teensy_index.json) to "additional board manager URLs".

Search for Teensy and install that package using the board manager (left pane of the main IDE window).

Set TOOLS→USB TYPE to SERIAL+MIDI.

(More detailed instructions on installing the Teensy toolchain are available here:[https://www.pjrc.com/teensy/td\\_download.html](https://www.pjrc.com/teensy/td_download.html).)

## Getting Source Code

Download the latest source code from <https://github.com/jimsnow/microtonal-controller> (I'd recommend the "git clone" method, as it makes downloading new updates easier, but downloading a zip file should work as well if you don't have git installed on your system.)

Open the file *microtonal-controller/src/microtonal-controller/microtonal-controller.ino* in the Arduino IDE.

At this point you can make any changes to the source code if you so desire.

Either way, you can connect the Mosaichord to your computer with a USB cable and click the right arrow button near the top left of the Arduino IDE window to compile and upload the firmware to the Teensy.

You may in some circumstances need to push the small white button on the teensy to put the bootloader into a mode where it waits for new firmware to load.

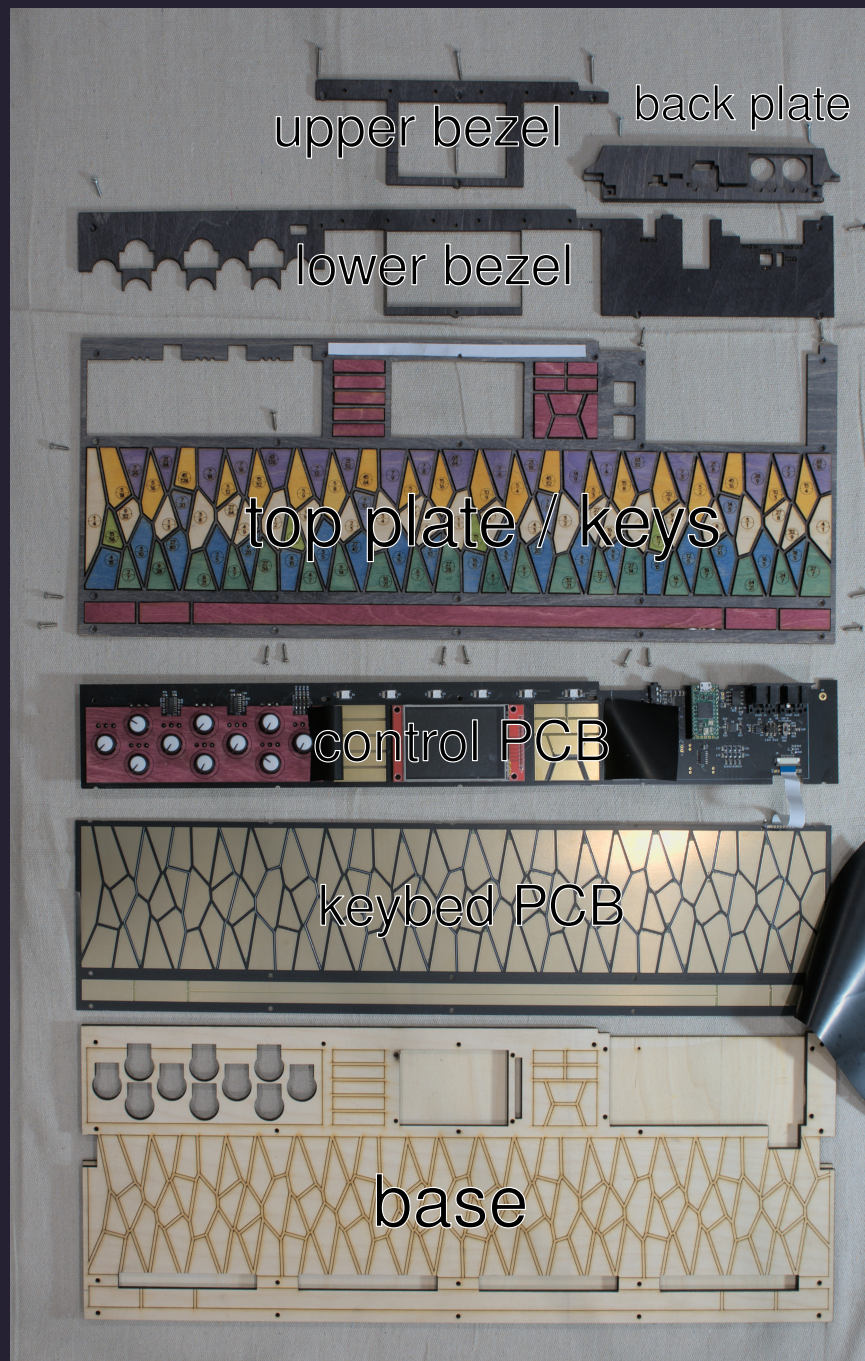
If all goes well, some red lights will flash on the Teensy and then it will reboot into the latest Mosaichord firmware.

## Debugging

One of the very useful features of the Arduino IDE is a serial console. This is used by the Mosaichord firmware to print diagnostic messages. It can be enabled in the IDE under “Tools→SERIAL MONITOR”.

Usually the output isn’t very verbose, but more output can be enabled with various options in the Mosaichord menus under SETTINGS→DEBUG. “SHOW RES” for instance displays current values being read by the 160 inputs to the ADC matrix (most of them correspond to keys, but some are for the control buttons, knobs, fixed calibration resistors, and the analog input headers), as well as resistance values before and after applying a calibration formula.

## Assembly and Disassembly



Sometimes it may be necessary to take the Mosaichord apart to fix something. Most often this will be to either clean oxidation off the printed circuit board (PCB) traces, causing some keys to be less responsive to pressure than they should, or to remove some foreign debris that managed to get caught between the PCB and force sensitive resistor film shorting a connection and causing a stuck note or button. These are both straightforward to remedy, but require disassembling the keyboard.

A small phillips screwdriver is all that's needed for assembly and disassembly, though some clear adhesive tape may come in handy if you need to replace any of the existing tape. Isopropyl alcohol is what we use to clean the PCB when necessary.

To disassemble the keyboard,

- disconnect from USB power
- remove two screws holding on the small back plate and remove it
- remove the 20 oval-head screws holding the bezel pieces and top plate in place
- gently pull the front panel off (the screen may be a tight fit)

And that's pretty much it. It isn't necessary to remove the knobs unless there's something wrong with the potentiometers or you need to replace the control PCB for some reason.

There are three force-sensitive resistor (FSR) pieces – one large one that covers the whole lower PCB, and then two smaller pieces on the control board to either side of the screen. This black plastic material<sup>2</sup> is called Velostat, and is typically used to make anti-static bags. It also has the useful property that it has a certain electrical resistance that reduces when pressure is applied, which makes it very useful as a force sensor.

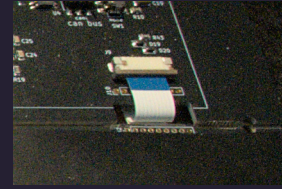
It is also possible to play the keyboard with nothing more than bare fingers directly on the PCB, but that requires very different calibration settings and results tend to vary with humidity.

The FSR should be held in place by a small amount of tape. If you remove the FSR to clean it or the PCB, be careful not to touch the exposed gold traces on the PCB under the FSR, as finger oils will eventually cause oxidation which will need to be cleaned off again some time in the future. We usually scrub the PCBs off with 70% isopropyl and paper towels, then let it dry and get all the lint off before taping the FSR back down and reassembling.

---

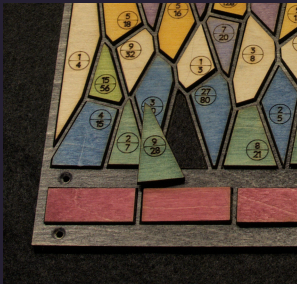
<sup>2</sup>Mouser part number 517-1704-36

If it ever becomes necessary to replace the upper or lower PCB, you'll need to disconnect the ribbon cable<sup>3</sup> between the two. This is easily done by pulling out the slide-lock mechanism on either PCB; the ribbon will come loose. The ribbon cable can be replaced by inserting the cable back in and then engaging the slide lock. (The ribbon goes in with the blue side up, as shown. Excess ribbon can be pushed out of the way under the upper PCB. The cable should not twist upside-down.



The Teensy 4.0 microcontroller is in a socket and can be carefully pulled out and replaced if necessary. (A Teensy 4.14 should work just as well, though it will overhang the socket by quite a bit and will require modification to the bezel or just leaving it off entirely. It's possible to modify another socket and solder it down to the pin holes provided to act as extra support. Remember to select the Teensy 4.1 board instead of 4.0 in the Arduino IDE when compiling/loading new firmware.)

The screen is soldered in place. It's possible to replace the screen using a hot-air rework station, but it's not easy.



The keys themselves are just birch ply attached to felt with a synthetic liquid hide glue. The felt is held in place by clear tape. If a key becomes detached from the felt (see figure), it can be re-attached by applying a thin layer of plain white glue on the wood and setting it on the felt where it goes, and letting the glue dry with something heavy set on top.

If the felt comes loose from the tape, then the tape can be cut away from the top plate with a hobby knife in the area affected and peeled carefully off of any affected keys, then

the tape can be replaced with packing tape.

Assembly is done in the reverse order as disassembly. Note that the four screws holding the upper bezel on are a little longer than the others, and the two holding the back plate on have a flat top rather than the oval top of the others.

It's usually a good idea to power up the Mosaichord and test it out once you have about three or four screws in, just to make sure the keys are all working right. It's much easier to identify problems and fix them before you've put all the screws back in place. I find it often takes two or three tries to get everything right when assembling a new Mosaichord.

---

<sup>3</sup>1mm pitch 9 conductor FFC cable with opposite-side contacts, Mouser part number 538-98267-0772

# Theory of Operation

It isn't necessary to know how the Mosaichord works to use it. However, some understanding may provide useful context for anyone who wants to make significant software modifications, and also some people just like to know how things work. Therefore we will explain the basic principles here. Some familiarity with basic electronics is assumed.

The schematic is not included here as it may change from time to time, but it can be found alongside the firmware source code.

## Keys

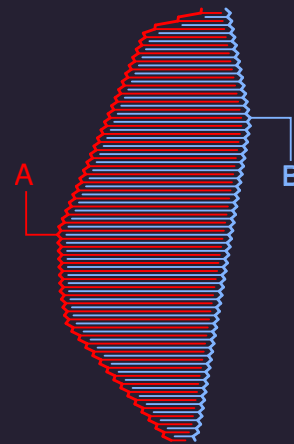
A good place to start is: how do the keys work?

Each key has a pressure sensor underneath it that normally has a fairly high electrical resistance. Pressing the key causes the resistance to drop – the firmer the pressure, the lower the resistance.

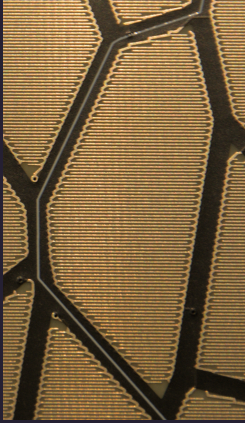
How these pressure sensors work is simple; on the printed circuit board we have two sets of gold-plated copper traces that are like fingers that nearly mesh but do not actually touch each other. Over the top we lay a force-sensitive resistor (FSR) material. The FSR connects both sets of traces (labelled A and B in the figure). The resistance between A and B is high when pressure on the FSR is light, but increasing pressure reduces the resistance, allowing electricity to flow more easily.

If we connect A to 3.3 volts through a pull-up resistor and connect B to ground, A will stay near 3.3 volts when pressure is light, but the low resistance path to ground will cause the voltage to drop much lower under high pressure.

The microcontroller we use has a number of analog input pins we can use to measure voltages, using one of several internal analog-to-digital converters (ADCs).



## Multiplexing inputs



Of course it's not quite as simple as wiring each key to a pin on the microcontroller. The keyboard has 113 keys in addition to a handful of buttons and knobs that we need to read as well. The microcontroller we use, the Teensy 4.0, only has 14 pins capable of analog input. Clearly we need more inputs than that.

There are a couple standard ways to deal with this, that have their pros and cons. One option is to use a key matrix. Keys are organized into a grid of rows and columns. You get a reading on a given key by measuring the resistance between its row and its column, with all other rows and columns dis-

connected from power and ground.

The matrix approach conserves on hardware, but it tends to suffer from “ghosting” where if you press three keys that are all corners of a common rectangle, the fourth corner will appear to be pressed as well. This problem can be reduced or eliminated by connecting each key through a diode, but that's a lot of diodes (eliminating some of the benefits of using a matrix to begin with) and diodes introduce a voltage drop — generally around 0.7 volts, or 0.3 volts for Schottky diodes. Since we're inferring pressure from the voltage we read, we need that to be as accurate as possible. Even 0.3 volts creates a pretty big “blind spot.”

Another approach is to use multiplexer chips. These are basically IO expanders that work like a selector switch that's controlled with some address pins. For example, a 16-way multiplexer would have 4 address pins used to encode a binary number that selects which of 16 input pins are connected to a single output. (Or vice versa. Multiplexers are usually bi-directional, and are often designed to work with analog signals.)

We think this is a good approach. It would probably work fine. It's not what we did, though – in the Mosaichord we use shift registers.

Normally, shift registers are used in digital systems to convert parallel data to serial (in an “input” shift register) or serial data to parallel (in an “output” shift register). The type we use is an output shift register, specifically the Texas Instruments TPIC6C595DR.

Output shift registers generally have an serial input pin and a clock input. (Actually, there are two clock inputs, but we drive them both at the same time from the same microcontroller pin.) There are 8 “regular” outputs, plus one extra.

A shift register can be thought of as an 8 bit memory device. Each output pin is either on or off depending on the stored bit, and pulsing the clock causes all the “bits” to shift over one place to the right, with the input pin determining whether the left-most bit is a 1 or a 0. If we need more than 8 bits, we can just connect the extra output of our first shift register to

32 keys sharing 1 adc pin (3rd key selected for reading)

The diagram shows a Teensy 4.0 microcontroller connected to a 3.3V supply and a 1k resistor. The ADC pin is connected to a common line that branches to the input of 32 keys, each represented by a TPIC6B595 shift register. The 3rd key is selected for reading, indicated by a red line. The clock signal is connected to the clock input of all shift registers.

What this allows us to do is connect one side of all the pressure sensor PCB traces together, pulled to 3.3 volts through a 1k pullup resistor. We connect the other side of the pressure sensor traces each to its own shift register pin.

The main loop of the program running on the microcontroller basically just scans all the keys in order over and over.

39

though only one key at a time per channel is productively doing anything, the rest can still potentially act as antennas and otherwise degrade signal quality.)

The upper PCB that has the microcontroller, I/O, and user interface controls has four shift registers, one for each ADC channel.

The seven-limit four-octave keybed has another 16 ADCs, 4 each per channel.

Effectively, that gives us 20 times 8 or 160 inputs. Not all of these are connected to keys. Some are connected to potentiometers, or fixed resistors, or in a few cases, nothing at all. Each ADC channel is responsible for 40 inputs.

We can do a complete scan of these 160 inputs about 500 to 900 times per second, depending on whether we care more about speed or accuracy.

One of the benefits of using shift registers as opposed to multiplexers is that there's no hard upper limit on the number of inputs we can have. That means we can use the upper "control" PCB with keyboards of practically any design, with more or less keys. We can even chain multiple of the keybed PCBs together (though we haven't actually tested this).

One feature of the keybed PCB is that the first shift register pin on each channel is connected to a fixed resistor. The purpose of this is that we can give each keybed PCB variation a different set of fixed resistors, so the control board can identify it. If the keybed isn't connected, the resistors will all show up as infinite resistance. Thus we can auto-detect when a new keyboard is connect to the chain simply by reading one bit past the end of the last shift register we know about and see if we read a resistor pattern we recognize.

Building a custom keyboard or expressive controller that chains off of the Mosaichord should be feasible as a DIY project, and in fact that's part of the reason why we describe in such detail how the keyboard works here.

## Autocalibration

One of the difficulties with the design as described above is that the four ADC channels aren't actually as independent from each other as we would like – readings on one channel tend to affect readings on other channels, sometimes significantly. The reason why this happens is that the force-sensitive resistor material creates a sort of electrical path between things that aren't *supposed* to be electrically connected, simply because they're physically near each other and they're both touching the FSR. What's worse is that the resistance of these unwanted electrical paths changes depending on what keys are being pressed and how hard.

We compensate for this by measuring the resistance between each of the four ADC channels on each keyboard scan. We then use the measured resistances to infer what the voltages

on each channel should be for that particular set of four inputs if the four channels weren't pulling each other towards some sort of average value.

This seems to work well enough. We may be able reduce the magnitude of the problem in the future by increasing the width of the thin strip of solder mask that separates the keys on the keybed PCB.

## Generating key pressure from MPE

Turning key pressure readings into MIDI commands (not to mention the autocalibration routines mentioned previously) is done by the firmware that runs on the microcontroller.

Basically, the main loop keeps track of whether the pressure of each key exceeds a certain minimum activation pressure – if so, it grabs an unused MIDI channel (or steals one, if none are available), and sends the appropriate pitch bend and note-on commands to begin the note.

On subsequent scans, if the key pressure remains above the minimum threshold, the program does not send additional note-ons, but rather it sends key pressure commands according to the MPE spec (if MPE key pressure is enabled) or it sends polyphonic aftertouch (if poly AT is enabled).

If the pressure drops below the minimum threshold, then a note-off is sent.

There are some additional details, depending on what settings are enabled: if dynamic velocity is enabled, then the program does not send an initial note-on until the key pressure exceeds the threshold for two keyboard scans. The difference between the pressure readings of those two scans is used to infer the velocity.

If tuning tables are enabled then we don't use pitch bend for microtuning at all – instead we search for the closest note to the one we want to play in our tuning table and send a note-on for *that* note.

## Other Components

There are a number of other hardware components connected to the microcontroller:

- a Texas Instruments PCM5102a i2s audio DAC with a stereo line-out jack
- a Texas Instruments SN65HVD230 CAN bus transceiver

- a pair of MIDI ports connected to serial IO pins on the microcontroller (with an optical isolator on the MIDI-in port, as required by the MIDI spec)
- a 240x320 touch screen with an ILI9341 controller (though touch screen functionality is unused due to a shortage of I/O pins) connected to microcontroller over SPI
- a string of 6 WS2812b programmable RGB LEDs

## Hardware Revisions

The Mosaichord uses two main printed circuit boards; an upper “control” PCB, and a lower “keybed” PCB.

### Control PCB Revisions

- Revision 1 (prototype): mostly functional, but some resistor changes required to prevent damage to ADC inputs, and DIN5 MIDI and LEDs can’t be used at the same time.
- Revision 2 (prototype): usable. Replaces a TL084 op-amp with TLV274, which works rail-to-rail. Added an audio DAC, but it does not function in this version because one of the pins that’s supposed to be grounded isn’t. Requires a “bodge wire” to connect some pressure pad traces that were accidentally left unconnected. (Otherwise, several of the menu buttons don’t work.) Rearranged serial IO pins so LEDs and serial MIDI can work at the same time.
- Revision 3 (production): Audio DAC now works. Re-arranged IO pins again to allow microcontroller to enable or disable pull-up on the four ADC lines. This allows for a calibration step that compensates for current leakage across adjacent keys in the keybed. Fixed unconnected pressure pad traces in revision 2.
- Revision 4 (production): Added 3.5mm socket for an expression pedal or a footswitch. No changes that directly impact firmware.

### Keybed Revisions

- Revision 1 (prototype): mostly functional, but 4/1 key does not work as it was accidentally left unconnected. Green PCB.
- Revision 2 (production): All keys work as intended.

- Revision 3 (production): Add soldermask strip between adjacent keys to increase distance between exposed traces that might interfere with each other. Change Identifying resistors from 10k, 0k, 0k, 0k to 10k, 1k, 1k, 1k as low-value resistors are somewhat more difficult to read accurately.

# Troubleshooting

| Symptom                                         | Cause                                                                                                 | Fix                                                                                                                       |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Electro-Mechanical                              |                                                                                                       |                                                                                                                           |
| continuous stuck note from external synthesizer | lost or garbled MIDI messages                                                                         | press NOTES OFF menu button to send a note-off command for all midi notes across all channels                             |
|                                                 | cable disconnected while note was playing                                                             | same as above after reconnecting keyboard with synth, or reset synth                                                      |
| key or menu button stuck on                     | FSR changes resistivity when in unusually warm environments                                           | move keyboard away from heaters or direct sunlight                                                                        |
|                                                 | overtightened screw                                                                                   | loosen screws near affected key or button, see if that resolves issue                                                     |
|                                                 | foreign debris on keyboard circuit board beneath force-sensitive resistor material                    | remove back plate and wooden front, including keybed, remove affected force sensitive resistor, clean both and reassemble |
| key or menu button has low sensitivity          | fingerprint or oxidation on printed circuit board traces underneath force-sensitive resistor material | same as above, but scrub affected traces with isopropyl alcohol                                                           |
| knob stuck at full value                        | broken solder trace                                                                                   | disassemble keyboard, reflow solder joints on affected pot, and reassemble or contact support                             |
| micro usb port on microcontroller broken        | mechanical stresses on usb cable or wear and tear                                                     | replace microcontroller (Teensy 4.0 with pins from pjrc.com) with current Mosaichord firmware loaded or contact support   |

|                                                        |                                                                       |                                                                                                                                                                             |
|--------------------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| light blinks on microcontroller when screen is redrawn | it just does that (Teensy 4.0 LED shares a pin with screen SPI clock) | it's normal                                                                                                                                                                 |
| wooden key top comes loose from felt backing           | wear and tear, or poor glue joint between wood and felt               | apply thin layer of white glue to underside of key, place key back on felt with heavy object on top, wait for glue to dry (if key is lost, contact support for replacement) |

| MPE / MIDI                                                         |                                                                           |                                                                                                                                                                                                                |
|--------------------------------------------------------------------|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| USB synthesizer not receiving MIDI commands                        | USB MIDI output is disabled                                               | enable USB MIDI under SETTINGS→MIDI→USB MIDI (enabled by default) or load device preset for a USB synth                                                                                                        |
|                                                                    | USB cable lost connection                                                 | unplug Mosaichord and re-establish connection; if USB connection is loose, try a different USB cable; if USB socket on microcontroller is damaged, replace Teensy 4.0 microcontroller and re-install firmware. |
| MIDI synthesizer not receiving MIDI commands over DIN-5 MIDI cable | din5 MIDI output is disabled                                              | enable din5 MIDI under SETTINGS→MIDI→DIN5 MIDI (disabled by default) or load a device preset for a non-USB hardware synth                                                                                      |
| only one every 15 or 16 notes sounds                               | MIDI synthesizer not in multi-timbral mode, only listening on one channel | consult synthesizer manual for instructions to put synthesizer in multi-timbral operating mode                                                                                                                 |
|                                                                    | MIDI synthesizer does not have multi-timbral capability                   | mono-timbral synths aren't generally supported by the Mosaichord keyboard                                                                                                                                      |
|                                                                    | MIDI synthesizer is a mono synth                                          | use an appropriate mono-synth device preset, or set the number of MIDI channels to 1 under SETTINGS→MIDI→CHANNELS                                                                                              |
| 15 out of every 16 notes sounds                                    | most likely the MIDI synthesizer reserves channel 10 for percussion       | update device preset to use skipChannel10Flag (requires firmware recompile/update) or set SETTINGS→MIDI→CHANNELS to 9 or less                                                                                  |

|                                                            |                                                                                        |                                                                                                                                                                                       |
|------------------------------------------------------------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| erratic changes in the sound of notes                      | not all channels are set to same patch                                                 | change patch preset from Mosaichord under PATCHES→PATCH, verify that the synth has loaded the correct patch on all channels                                                           |
|                                                            | not all channels have same settings                                                    | reconfigure the synth to have all parameters the same for all channels, save configuration as a preset if necessary, (reloading all the patches or resetting the synth may also work) |
| tuning seems slightly off (sounds almost like 12-EDO)      | pitch bend range on the synth is much wider than the Mosaichord thinks it is           | set pitch bend range on Mosaichord to match synth under SETTINGS→MIDI→BEND RANGE or reconfigure synth to match Mosaichord's bend range configuration                                  |
|                                                            | effects or synth settings are obscuring sense of definite pitch                        | try disabling oscillator detune, chorus, vibrato, or tremolo effects                                                                                                                  |
| tuning is pretty far off                                   | pitch bend range on the synth is much narrower than the Mosaichord thinks it is        | set pitch bend range on Mosaichord to match synth or reconfigure synth to match Mosaichord                                                                                            |
|                                                            | tuning table-based preset in use, but Mosaichord and synth don't agree on tuning table | set correct device preset for synth, and set tuning table manually on synth if necessary                                                                                              |
| tuning is off, but only for some patch presets             | some patch presets have a different pitch bend range than the others                   | set pitch bend range manually on synth or Mosaichord so they match                                                                                                                    |
|                                                            | some patch presets apply a non-12-EDO temperament                                      | override patch's temperament settings for that patch if possible on that synth                                                                                                        |
| notes cut out too quickly, have awkward-sounding envelopes | note-on attack and key pressure volume may be interfering with each other              | turn off either pressure or velocity under SETTINGS→CONTROLS                                                                                                                          |

|                                                          |                                                                                                                                |                                                                                                                                                                                                                       |
|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                          | polyphony limit exceeded                                                                                                       | if possible, increase number of MIDI channels in use under SETTINGS→MIDI→CHANNELS, otherwise keep note release time short and be careful not to play more notes at once than your synthesizer can play simultaneously |
| reverb cuts off too soon                                 | some synths apply reverb <i>before</i> volume control (CC 7 usually), which we often use to control per-note volume expression | not much we can do about this directly, other than to not modulate CC 7 with key pressure, or disable reverb entirely on the synth and route its output through an external reverb unit                               |
| noticeable key pressure lag                              | controller may be sending MIDI messages too fast for synthesizer to process                                                    | increase SETTINGS→CONTROLS→P BACKOFF until latency goes away (“pressure backoff” is the time interval between sending pressure updates in microseconds).                                                              |
| MPE synth does not recognize Mosaichord as an MPE device | synth may have missed the mpe initialization that happens when keyboard is booted or mpe device preset is loaded               | re-send mpe initialization (SETTINGS→MIDI→MPE INIT) or re-load one of the MPE device presets                                                                                                                          |
| Seldom-Used Features                                     |                                                                                                                                |                                                                                                                                                                                                                       |
| unable to send/receive CAN bus messages                  | incorrect cable termination                                                                                                    | verify 120 ohm resistor at both ends of CAN bus, turn on CAN bus resistor switch if needed                                                                                                                            |
|                                                          | no software handling                                                                                                           | add desired software CAN messaging support to firmware source code, recompile and load                                                                                                                                |